

Entwicklungsprojekt 2021/22

von Vincent Luciano und Luzie Ewert



Inhaltsverzeichnis

Alleinstellungsmerkmal.....	Folie 3
Zielhierarchie.....	Folie 4
Risikoanalyse.....	Folie 5
Architekturdiagramm.....	Folie 6
PoC: Jemanden finden, der die gleiche Strecke fährt.....	Folie 7
PoC: Wie sich die Personen in der Bahn/ im Bus finden.....	Folie 8
PoC: Wie endet eine Fahrt.....	Folie 9
PoC: Ticketgeber dazu bringen, die App zu nutzen.....	Folie 10
Code - PoC: Jemanden finden, der die gleiche Strecke fährt.....	Folie 11
Code - PoC: Jemanden finden, der die gleiche Strecke fährt.....	Folie 12
Code - PoC: Jemanden finden, der die gleiche Strecke fährt.....	Folie 13
Code - PoC: Jemanden finden, der die gleiche Strecke fährt.....	Folie 14
Code - PoC: Wie sich die Personen in der Bahn/ im Bus finden.....	Folie 15
Code - PoC: Wie endet eine Fahrt.....	Folie 16
Projektplan.....	Folie 17

- **Einmalig, gibt es so noch nicht auf dem Markt**
- **Bietet soziale Interaktion im Alltag**
- **unterstützt soziale Bereitschaft der Menschen**
- **unterstützt sozial Benachteiligte**
- **stellt Verbindung zwischen Personen mit Ticket und Personen ohne Ticket her, die sich nicht kennen**



Da sich unsere Use Cases, welche wir als PoC's darstellen wollen, noch einmal geändert hatten, wollten wir auch die beziehungsweise das Alleinstellungsmerkmal dementsprechend anpassen. Mit Veränderung der PoC's veränderte sich auch der Fokus unserer Herangehensweise und somit auch das Alleinstellungsmerkmal. Schwerpunkt war es daher nun, die beiden Personen miteinander zu verbinden, durch unsere Anwendung und ohne, dass diese sich vorher kannten. Da es, wie bereits in einem der anderen Punkte aufgegriffen, keine ähnliche Anwendung gibt, die diesen Service anbietet, stellt dies auch unseren Schwerpunkt dar und bildet das Hauptalleinstellungsmerkmal.

Zielhierarchie

Strategische Ziele

- Umwelt schonen
- Weniger Schwarzfahren
- Mehr Kontakt zwischen Menschen
- Jeder Mensch kann fahren
- Dem Nutzer einfache Bedienung ermöglichen
- Weitflächiges Netzwerk

Taktische Ziele

- Menschen dazu bringen die Anwendung zu nutzen
- Eine Community Bauen

Operative Ziele

- Einen Prototyp bauen
- Einen MVP bauen
- Erste Nutzer bekommen und Werbung
- Feedback bekommen
- stetige Updates, zur Verbesserung der Usability



In der Zielhierarchie sollte es vor allem darum gehen, die Alleinstellungsmerkmale möglich zu machen. In der Nachbearbeitung wurden die Punkte dessen nochmal angepasst und es wurde geschaut, in wie fern die als Proof- of- Concept dargestellten Use Cases Bedingungen an die Zielhierarchie stellen und durch welche operativen Ziele man diesen entgegenwirken kann. Da wir in unseren vorherigen PoC's bereits diese Überlegungen angestellt hatten und uns nun auf kleinere Use Cases beschränkt hatten, hatten wir bereits einige Ziele abgedeckt. Um allerdings den PoC's entgegenzukommen, mussten ein paar wenige Aspekte, wie das ausbauen der Usability in der Zielhierarchie nochmals betont werden und wurde dementsprechend und den statischen und operativen Zielen nochmals aufgegriffen.

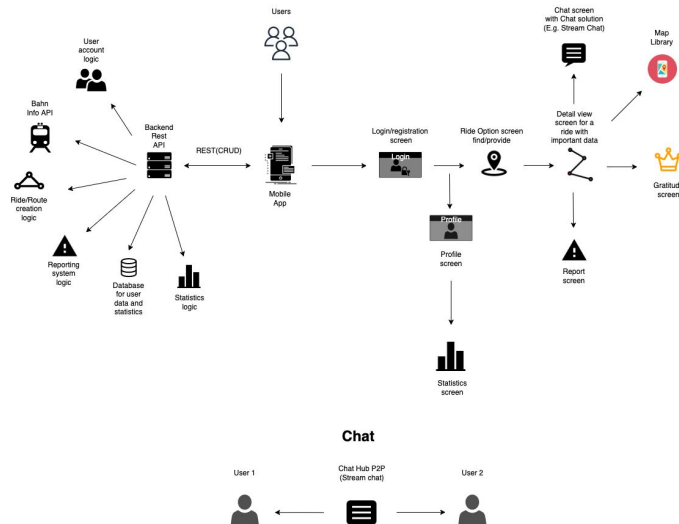
Risikoanalyse

- **Passende API finden**
- **APIs könnten ausfallen**
- **Fehlerhafte Entscheidung der Programmiersprache**
- **Netzwerk kreieren - setzt viele Nutzer voraus**
- **Wir setzen auf die Solidarität der Menschen**
- **Entstehung von spät erkannte Bugs in der Entwicklungsphase**
- **Ausfall einer der Teammitglieder**
- **Rechtliche Konflikte mit den Verkehrsunternehmen**
- **Genug Nutzer erreichen**
- **Große Internetabhängigkeit**



Nachdem die nicht technischen Risiken bereits erörtert wurden und die technischen nun in Form von PoC's dargestellt werden sollten, ging es auch darum sich Gedanken über die Technologien der Anwendung zu machen und welche da am besten für unser Vorhaben geeignet sind. Hierfür wurden zwei neue Punkte der Risikoanalyse hinzugefügt, welche als Risiko abgewägt werden mussten und wo entsprechend dem minimalsten Risiko hingehend gehandelt werden sollte. Diese Aspekte traten unter anderem auch während der Spezifizierung der PoC's auf, da wir dabei bemerkten, dass das Bilden eines Netzwerks und die Suche nach einer anderen Person nicht ohne eine stabile Internetverbindung möglich sein würde. Dieses Risiko galt es abzuwägen und nach Alternativen zu schauen. Außerdem mussten wir einschätzen, wie sehr eine instabile Internetverbindung die Anwendung einschränken würde und an welchen Stellen man versuchen könnte, auf eine nicht Internet abhängige Alternative umsteigen zu können.

Architekturdiagramm



Das Architekturdiagramm ließ sich aus der vorherigen Arbeit ableiten, in dem in der Risikoanalyse überlegt wurde, für welches Endgerät die Anwendung letztendlich entwickelt werden sollte und wodurch man die meisten Use Case's abdecken kann. Da wir beschlossen eine dynamische Suche möglich zu machen und dies unser meist genutzter Anwendungsfall sein würde, fiel die Wahl relativ schnell von einer Web Anwendung auf eine App umzusteigen. Hierbei werden REST API und User Interface getrennt behandelt, um so viel Datenverkehr einzusparen wie möglich und damit die Nutzer stets bei der Nutzung der App nur auf die Informationen zugreifen müssen die bereits vom System bereitgestellt werden und die Abfragen im Hintergrund laufen können. Was der Nutzer am Ende im UI zu sehen bekommt, sind also nur die Screens, welche die Informationen überbringen. Jegliche API's und unsere Datenbank sollen somit im Backend ausgelagert werden. Hierbei war es uns wichtig, alle nötigen Funktionen, für die Umsetzung der PoC's aufzuführen. Auch Funktionen, wie das Report System wurden im Architekturdiagramm dargestellt, um eine Vorstellung davon zu bekommen, wie die App im besten Falle aussehen würde. Allerdings sind einige Funktionen, wie dem Report System zum Beispiel, nicht

unbedingt im Projekt eingeplant und werden in diesem Umfang nicht umgesetzt werden können, damit wir uns auf die wichtigsten Funktionen konzentrieren können.

Die Architektur des Chats ist ebenfalls noch einmal unterhalb des Projektplans modelliert. Geplant ist einen Peer- to- Peer Chat zu integrieren, da diese einfach zu Beschaffen sind und keine permanenten Chats erstellt werden sollen, also die Nachrichten auch nicht permanent gespeichert werden sollen.

PoC: Jemanden finden, der die gleiche Strecke fährt

Exit-Kriterien:

- Nutzer eigene Fahrt in App eingeben können über Karte
- Person mit übereinstimmender Fahrt finden
- Gemeinsame Fahrt vereinbaren

Fail-Kriterien:

- schlechte Internetverbindung
- kein Zugriff auf Live- Daten der ÖPNV
- gestörtes GPS Signal

Fallbacks:

- Informationen der ÖPNV manuell als kompaktere Version eingeben
- unvollständige Datensätze können durch Algorithmen angepasst werden
- Live Daten werden durch die Eingabe der Nutzer ersetzt



Als erster Use Case, welchen wir mittels eines Proof- of- Concept's auf die technische Umsetzung prüfen wollen, filterte sich recht schnell unser ebenfalls genanntes Alleinstellungsmerkmal heraus, wie sich eine Person ohne Ticket und eine Person mit Ticket überhaupt finden. Es ging darum, zu überlegen, aus welchen Gründen sich diese zwei Personen zusammen tun würden und wie der Prozess des Findens stattfinden würde. Die beiden Personen hätten somit als Schnittstelle die gemeinsame Richtung in die sie fahren und aus welcher sie kommen. Da wir hier sehr abhängig von den öffentlichen Verkehrsmitteln und deren Linien sowie Abfahrtszeiten sein würden, musste zuerst ein Weg gefunden werden, an diese Informationen zu kommen und diese auch ständig aktualisieren zu können. Da einige Web Services diese Informationen bereits nutzen und auch zur Verfügung stellen, kam die Überlegung auf, eine API zu nutzen, über welche wir ebenfalls die nötigen Informationen bekommen würden.

Nach einigem Abwägen wurde ebenso über den Zeitpunkt entschieden, wann sich die Personen in der Regel nach einer Mitfahrgelegenheit umschauen würden, und inwiefern dies Einfluss auf die Nutzung der Anwendung haben würde. Da eine Bahn- oder Busfahrt meist von

vielen Faktoren abhängig ist und nur selten weit im Voraus geplant wird, entschieden wir uns dazu, eine dynamische Suche möglich zu machen. Das bedeutet, dass der Nutzer in der Lage sein sollte, zu jeglichem Zeitpunkt, nach einer Mitfahrgelegenheit suchen zu können. Um jeden Fall abdecken zu können und wenige Ausfälle geplanter Fahrten zu garantieren, sollte somit der Zeitpunkt der Suche kurz vor einer Fahrt stattfinden können. Dies hat zum Vorteil, dass die beiden Personen mit großer Wahrscheinlichkeit die geplante Bahn oder den geplanten Bus/Zug auch bekommen und es sich als einfacher gestaltet, eine Fahrt kurz vorher zu planen, ohne in Zeitstress zu geraten, auf Grund von eigenen Verzögerungen oder ähnlichem.

Um dem Nutzer so viel Arbeit abzunehmen, wie möglich, sollte auch die Bedienung zum Eingeben der Fahrtinformationen so einfach wie möglich gestaltet sein. Damit der Nutzer also nicht alle Daten manuell eingeben muss und wir die Koordinaten der Stationen ja bereits durch die vorhandenen API's bekommen würden, bietet es sich an, auf die GPS Daten der Nutzer zuzugreifen. Darüber können sowohl die am nächsten liegenden Stationen ermittelt werden, als auch der Standort des Nutzers was für den weiteren Verlauf von Vorteil ist. Da das GPS Signal meist beständig ist in der Stadt bietet es einen sicheren und einfachen Weg.

Bei einer erneuten Iteration hat sich die Gruppe bei den Exit- Kriterien darauf konzentriert, zu erklären, was die letztendliche Ausgangssituation sein soll. Wo vorher noch darüber gesprochen wurde, wie diese Exit- Kriterien erreicht werden sollen, wurde nun in dem Punkt Exit- Kriterien auf folgende drei Kriterien gekommen, welche zuvor umschrieben wurden. Um eine Person also zu finden, welche die gleiche Strecke fährt schien es vorerst sinnvoll, dass eine Person in der Lage ist, überhaupt der App die Information zu übergeben, wo diese Person hinfahren möchte und welches Transportmittel dafür gewählt wird, damit diese Informationen später weiter verarbeitet werden können. Als Exit- Kriterium sollte die App die Eingabe dieser Informationen ermöglichen und dem Nutzer dabei so viel Arbeit abnehmen wie möglich. Außerdem sollten die Personen, um zusammen fahren zu können, die gleiche Strecke mit dem gleichen Verkehrsmittel tätigen wollen. Auch hierfür wurde sich dafür entschieden, dass die

Applikation diese Aufgabe übernehmen sollte und dem Nutzer so wenig Aufwand wie möglich bereitet. Anschließend sollte eine Verbindung der Nutzer eingegangen werden, auf welche sich beide Seiten einigen, da der Fall eintreten könnte, dass eine der beiden Nutzer nicht in der Lage sein könnte oder sich umentscheiden könnte eine gemeinsame Fahrt zu starten.

PoC: Wie sich die Personen in der Bahn/ im Bus finden

Exit-Kriterien:

- Personen sind in der Lage, sich über eine Kartenansicht und den GPS Standort zu finden
- Können über einen Chat kommunizieren und genauen Standort mitteilen
- Können zudem, beim Erstellen einer Fahrt, durch betätigen eines Buttons mitteilen (bei Bahn oder Zug), in welches Abteil/welchen Wagon sie einstiegen

Fail-Kriterien:

- Schlechte Internetverbindung
- Erfolgreiche Implementation der Einstiegsbereich Abfrage kann durch viele verschiedene Faktoren scheitern
 - unterschiedliche räumliche Aufteilungen bei Bahn, Bus, Zug
 - Menschliche Fehler: indem eine Änderung des geplanten Einstiegsbereichs vergessen wird

Fallbacks:

- Explizit die Chat Funktion nutzen



Bei dem Use Case, wie sich zwei Personen, die bereits ein "Match" gebildet haben, da sie die gleiche Strecke fahren, wurde zuerst über eine Möglichkeit nachgedacht den Datenverkehr möglichst klein zu halten. Durch wenige Klicks sollte der Nutzer in der Lage sein, die andere Person finden zu können. Die offensichtlichste Möglichkeit war für uns vorerst einen Chat einzubauen, durch den die Nutzer sich nach bilden eines "Matches" vor Ort finden können, ob in Bus/Bahn oder an der Haltestelle. Allerdings ging es auch zu bedenken, dass einige Anhaltspunkte sich überschneiden und somit durch andere Mechanismen kommuniziert werden könnten. So könnte man eine Angabe, mittels eines Buttons, seitens der Ticket besitzenden Person, vorab geben, in welchen Teil der Bahn man zum Beispiel einsteigen möchte. Zudem kam die Überlegung auf, ob Profilbilder den beiden Personen das Finden ebenfalls vereinfachen würde, da sie so wüssten, nach wem sie suchen müssen.

Ein weiterer Punkt, welcher das Finden deutlich vereinfachen würde, wäre es, wenn die Personen den Standort des jeweils anderen immer einsehen könnten. Mittels GPS Ortung und einer damit zusammenhängenden Kartenansicht könnte man eine solche Funktion

ermöglichen. Da wir bereits zuvor schon überlegt hatten GPS in unserer Anwendung zu integrieren, würde dies keinen neuen Schwierigkeitsfaktor darstellen und könnte nur für diese Zwecke weiterverwendet werden.

Da es sich als schwierig gestalten könnte, eine fremde Person in mitten von anderen fremden Personen zu finden, wurde sich auf mehrere Exit- Kriterien geeinigt, durch welche diese Suche vereinfacht werden sollte. Einerseits war es die Idee, über den Standort eine Suche zu generieren, da so einiges an Informationsaustausch zwischen den Nutzern abgenommen werden würde und diese sich über einen Chat lediglich freiwillig oder zur genauen Beschreibung des Outfits, etc. (zur besseren Identifikation) verständigen könnten. Trotzdem erschien, wie bereits erwähnt, ein Chat als unabdingbar, da so jeder Fall abgedeckt werden würde, in dem, wenn zwei Personen beispielsweise an einer sehr vollen Station stehen, diese sich immer ausfindig machen können. Als zusätzliche Hilfe und für den Fall, dass eine Person bereits in Bahn oder Zug sitzt, sollte ein Button zur Angabe der Sitzposition im Verkehrsmittel, der dazu steigenden Person mehr Information über den Aufenthalt der anderen Person geben.

PoC: Wie endet eine Fahrt

Exit-Kriterien:

- App ist in der Lage, Fahrt zu tracken und gemeinsame Fahrt größtenteils selbstständig zu beenden nach Erreichen des Ausstiegspunktes
- Personen sind in der Lage Fahrt manuell zu beenden

Fail-Kriterien:

- schlechte Internetverbindung
- schlechtes GPS Signal
- Kein passendes Live ÖPNV Daten API

Fallbacks:

- statischer Button ersetzt GPS Funktion



Da eine Person auf ihr Ticket nur eine weitere Person mitnehmen kann, kann auch nur eine Person mit einer anderen Person auf einmal ein "Match" eingehen. Um eine neue Person annehmen zu können müsste die Person mit Ticket somit in der Lage sein, die Fahrt beenden zu können. Auch andere Situationen in denen sich eine der Personen unwohl in der Situation fühlen könnte und die Fahrt vorzeitig beenden möchte könnten somit abgedeckt werden. Auch hier wollten wir vermeiden, dass der Nutzer ständig auf die Anwendung zugreifen muss und diese so viel automatisch ausführen kann, wie möglich. Wenn man sich die Deutsche Bahn App nun anschaut, sieht man, dass sobald eine Fahrt eingegeben wurde, man auf dem Screen sehen kann, wo die Bahn gerade ist und um welche Uhrzeit sie sich an der gewünschten Ausstiegsstation befinden wird. Durch die API's, die von der Deutschen Bahn zur Verfügung gestellt werden, hatten wir gehofft, diese Informationen zu bekommen und zu nutzen, um, wenn die Person ohne Ticket einen Ausstiegspunkt auswählt, die Fahrt der Bahn oder des Busses so zu tracken, dass, sobald die Bahn oder der Bus diese Station erreicht, die Fahrt automatisch beendet wird.

Um dem Nutzer trotzdem noch die Möglichkeit zu lassen, die Fahrt

vorzeitig zu beenden, oder, falls die API das trocken der Busse und Bahnen nicht ermöglicht, auf eine manuelle Alternative umsteigen zu können, soll es ebenfalls einen Button auf dem Personen mit Ticket UI geben, der dies ermöglicht.

Wie bereits erwähnt, sollte der Nutzer dazu in der Lage sein können, eine Fahrt zu beenden, um eine neue Fahrt beginnen zu können, falls gewünscht oder nötig. Als Exit- Kriterium wurden sich hierbei zwei Möglichkeiten ausgedacht, dies zu tun, um einerseits dem Nutzer wieder so viel Arbeit wie möglich abzunehmen aber auch andererseits um ihm trotzdem noch die freie Entscheidung zu lassen, eine Fahrt vorzeitig zu beenden, da genügend Edge Cases gefunden wurden, die diese Funktion erfordern.

PoC: Ticketgeber dazu bringen, die App zu nutzen

Exit-Kriterien:

- Person mit Ticket wird das Gefühl gegeben, etwas Gutes getan zu haben
 - durch Statistiken und motivierende Worte
- Anreiz wird gesteigert, App wieder zu nutzen, um Gefühl wieder zu bekommen

Fail-Kriterien:

- Datenschutz
 - falls viele Nutzer nicht zustimmen, dass deren Daten benutzt werden können, ist eine erfolgreiche Implementation nur bedingt möglich

Fallbacks:

- ausschließlich Push- Benachrichtigungen versenden



Da wir aus rechtlichen Gründen eine Bezahlung nicht möglich machen können, musste ein neuer Anreiz gefunden werden, weshalb die Personen mit Ticket die Anwendung nutzen wollen. Bei unserer Recherche und einem Brainstorming haben wir uns Gedanken über andere Apps und Web Anwendungen gemacht die dem Nutzer keinen Eigennutzen bieten, jedoch trotzdem tausende von Menschen dazu überzeugen sie zu nutzen. Dabei sind wir unter anderem auf Spenden-Apps wie ShareTheMeal gestoßen. Bei dieser App geht es darum regelmäßig kleine Spenden an Projekte seiner Wahl zu spenden, bei denen kein monatlicher Beitrag abgezogen wird sondern der Nutzer je nach beliebigen die App öffnen und durch einen Klick spenden kann. Hierbei wird durch Statistiken gezeigt wie viele Menschen zum Beispiel bereits bei welchem Projekt geholfen haben. Ein ähnliches Konzept sollte auch bei unserer App den Nutzer dazu bringen und motivieren die App weiterhin zu nutzen und diesem ein gutes Gefühl überbringen. Daher überlegten wir uns ebenfalls mit Statistiken, sofern dies mit dem Datenschutz vereinbar ist und zudem, durch positiven Zuspruch, in Form von Push- Benachrichtigungen oder ganzen Screens, nach Abschließen einer Fahrt, zu arbeiten.

Bei der Iteration wurde sich bei den Exit- Kriterien vor allem darauf konzentriert, mit welchen Anreizen und Gefühlen der Nutzer gearbeitet werden soll, um eine attraktive App zu kreieren. Ziel ist es somit, durch die gewählten Funktionen einen Einblick, mittels Statistiken, in den Einfluss der Nutzung des Nutzers zu geben und durch gewählte Screens ein positives Gefühl beim Nutzer auszulösen. Da das Einsetzen einer Bezahlung aus rechtlichen Gründen nicht oder nur schwierig umsetzbar erscheint, kamen die Emotionen des Nutzers am ehesten in Frage, da einige positive Beispiele zeigen, dass auch dies ein effektives Mittel ist, um Nutzer dazu zu gewinnen.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt
(Haltestellen Suchen Pseudo Code)

```
const getAllStationsInRadius = async (geoLocation) => {  
  await axios.get('http://localhost:8001/TicketFor2/stationsByGeoLocation/' + geoLocation)  
    .then((res) => {  
      setStations(res.data)  
    })  
    .catch((err) => {  
      console.log(err);  
    });  
}
```

```
const getStationsInRadius = async (req, res) => {  
  await axios.get('http://transit-api/transit/' + req.geoLocation)  
    .then((response) => {  
      return res.status(200).json({ stations: response.data });  
    })  
    .catch((err) => {  
      console.log(err);  
    });  
}
```

Da einige Abhängigkeiten für dieses Projekt benötigt werden, um die letztendlichen Exit- Kriterien der Proof- of- Concepts zu erfüllen, musste, mangels der Zeit und einiger Komplikationen bezüglich der Erschaffung der benötigten Daten, auf Mock- Ups für den ersten Rapid Prototype zugegriffen werden. Ein Beispiel dafür ist die Suche nach Haltestellen in der Nähe, auf der Karte. Da für diesen Prozess die genauen Koordinaten benötigt werden, wo sich sämtliche Haltestellen befinden, sollten diese aus einer bereits vorhandenen Datenbank oder einem bereits vorhandenen Objekt, welches diese Informationen bereits geordnet und den entsprechenden Linien zugeordnet, beschaffen werden. Plan war es, diese Informationen von der HERE API zu bekommen und sie anschließend über Request ansprechen zu können. Wenn die Person nun den MapScreen öffnet, sollen nur jene Stationen angezeigt werden, die sich in unmittelbarer Nähe befinden. Mittels eines eingegebenen Radius soll dies ermöglicht werden, wie in dem oben gezeigten Beispiel.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Fahrt Erstellen)

```
const createRide = async (rideData) => {
  await axios.post('http://localhost:8001/TicketFor2/ride', rideData)
    .then((res) => {
      const newRide = res.data.ride
      setRide(newRide)
      handleRideCreationSuccess()
    })
    .catch((err) => {
      console.log(err);
    });
};
```

```
const createRide = ((req, res) => {
  const newRide = new Ride({...req.body})
  newRide.save()
    .then((ride) => {
      return res.status(200).json({ ride: ride });
    })
    .catch(err => {
      return console.log(err)
    });
});
```

Wenn eine Fahrt von der Person mit Ticket erstellt wird, sollte sichergestellt werden, dass die Informationen für die Personen ohne Ticket in der Nähe sichtbar gemacht werden. Hierfür wurde sich dazu entschieden, die Daten der erstellten Fahrt vorerst in der Datenbank abzuspeichern, damit sie später von anderen Devices aus abgerufen werden kann. Dieser Vorgang ist in den obigen Code Snippets abgebildet, da er einen fundamentalen Vorgang für die Verbindung zweier Personen darstellt.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Fahrten Suchen Pseudo code)

```
const getAllRidesByGeolocation = async (geoLocation) => {  
  await axios.get('http://localhost:8001/TicketFor2/ridesByGeoLocation/' + geoLocation)  
    .then((res) => {  
      setRides(res.data)  
    })  
    .catch((err) => {  
      console.log(err);  
    });  
}
```

```
const getRidesByGeolocation = (req, res) => {  
  const kmToRadian = function(km){  
    const earthRadiusInMiles = 6378;  
    return km / earthRadiusInMiles;  
  };  
  Ride.index({ location : "2dsphere" }, { expireAfterSeconds: 3600 })  
  const query = {'location' : {'$geoWithin': { '$centerSphere': req.geoLocation, kmToRadian }}}  
  
  Ride.find(query, (err, result) => {  
    if(err) res.send(err)  
    else res.send(result)  
  })  
}
```

Die Stationen, von welchen eine, von der Person mit Ticket erstellte, Fahrt ausgeht, werden dem Suchenden dann in einem bestimmten Radius, welcher vorher berechnet wird, gezeigt, je nachdem, ob eine Person mit Ticket zuvor eine Fahrt von einer Haltestelle in der Nähe eingegeben hat.

So kann garantiert werden, dass die suchende Person sich auf jeden Fall in der Nähe der Station befindet und sie bereits direkt alle möglichen Mitfahrgelegenheiten angezeigt bekommt und nicht nach eigener Eingabe erst feststellt, dass keine Person die gleiche Strecke, wie die suchende Person, fährt. Wie bereits zuvor angesprochen, soll innerhalb eines festgelegten Radius geschaut werden, ob sich erstellte Fahrten von Personen mit Ticket in der Nähe befinden. Um das zu ermöglichen sollten zuerst einmal die GPS Daten der suchenden Person zur Verfügung stehen um dann, anhand dieser, einen Radius ausrechnen zu können, mittels dessen die erstellten Fahrten als Marker auf der Karte zu sehen sein werden. Der Radius ist zudem deshalb von Vorteil, dass so eine dynamische Suche garantiert werden kann, da Personen nur unmittelbar vor einer Fahrt diese erstellen und auch als suchende Person einsehen und anfragen können. Hierfür gibt es einige

Formeln die man zur Berechnung des Radius nutzen könnte, um anschließend die angebotenen Fahrten innerhalb des Radius als neues Objekt in der Karte abrufen und als einzelne Marker abbilden zu können.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Fahrt Aktualisieren)

```
const updateRide = async (rideData) => {  
  await axios.put('http://localhost:8001/TicketFor2/ride/' + ride._id, rideData)  
    .then((res) => {  
      const newRide = res.data  
      setRide(newRide)  
      setShowConfirmation(false)  
    })  
    .catch((err) => {  
      console.log(err);  
    });  
};
```

```
const updateRide = ((req, res) => {  
  Ride.findByIdAndUpdate(req.params.id, {...req.body}, {new: true},  
    (err, result) => {  
      if(err) res.send(err)  
      else res.send(result)  
    })  
})
```

Um sowohl der Person mit Ticket, als auch der Person ohne Ticket, die einzelnen Vorgänge sichtbar zu machen, also dass eine Fahrt erstellt von der Person mit Ticket erstellt wurde, eine Mitfahrgelegenheit von der Person angefragt wurde, diese dann wiederum von der Person mit Ticket bestätigt wurde und die Fahrt abschließend abgeschlossen oder vorzeitig abgebrochen wurde (Created -> Pending -> Started -> Completed/Canceled), müssen auch diese Informationen ständig an das Backend übergeben werden, damit sie vom jeweils anderen Client abgerufen werden können. Auch hierfür wurde mit Updates an die Datenbank gearbeitet, um diese der anderen Person auf einfachem Wege vermitteln zu können und eine bessere Performance zu garantieren.

Code - PoC: Wie sich die Personen in der Bahn/ im Bus finden

```
const [messages, setMessages] = useState([]);
```

```
const onSend = useCallback((messages = []) => {  
  setMessages(previousMessages => GiftedChat.append(previousMessages, messages))  
}, [])
```

```
<GiftedChat  
  messages={messages}  
  onSend={messages => onSend(messages)}  
  user={{  
    _id: 1,  
  }}  
</>
```

Für den Rapid Prototype musste, wegen mangelnder Zeit, auf den geplanten Fallback zurückgegriffen werden, welcher das Finden der beiden Personen, die zuvor eine gemeinsame Fahrt bestätigt haben, ermöglicht. Der geplante Fallback war es somit, einen Chat zu integrieren, durch welchen sich die beiden Personen über ihren genauen Aufenthalt austauschen können. Auch äußere Merkmale können im Chat beschrieben werden, da sich die Personen höchstwahrscheinlich noch nie zuvor gesehen haben, und somit nicht wissen können, nach welcher Person sie überhaupt suchen müssen. Hierbei wurde sich vorerst für einen Gifted Chat entschieden, welcher einen Chat erstmals simuliert. Im Main Prototype soll dann mit dem Stream Chat gearbeitet werden, da dieser bereits ein eigenes Backend hat und dies einiges an Zeit spart, welche nur begrenzt zur Verfügung steht. In den gezeigten Snippets ist somit zu sehen, inwiefern Nachrichten erstellt werden und “versendet” werden.

```
const handleRideStatusChange = (status) => {  
  updateRide({ ride_status: status }).then(() => {  
    setRide({...ride, ride_status: status })  
    navigation.navigate('PostRideScreen')  
  })  
  .catch(err => {  
    return console.log(err)  
  });  
}
```

Um eine Fahrt zu beenden, sollte abermals für die Umsetzung auf einen geplanten Fallback zurückgegriffen werden, da das Beenden einer Fahrt, im Exit- Kriterium, sowohl durch Betätigen eines Buttons als auch durch den Abgleich der GPS Koordinaten der beiden Personen, sowie einem damit zusammenhängendem Radius- Entfernungsmesser, möglich sein sollte. Im Rapid Prototype ist das Beenden einer Fahrt allerdings vorerst nur durch Betätigen eines Buttons möglich, da die Umsetzung des Live Tracking der GPS Daten bis zu diesem Zeitpunkt noch nicht durchgeführt werden konnte. Daher wird bei Betätigen des “Fahrt beenden” Buttons der Status der Fahrt verändert und beide Personen werden wieder auf den nächsten Screen geführt, welcher beiden Personen zu einer gelungenen Fahrt gratuliert. Anschließend haben beide Personen die Möglichkeit wieder auf den MapScreen zurückzukehren, damit die Möglichkeit besteht im Anschluss direkt eine neue Fahrt starten zu können.

Projektplan

Meilenstein	Zeitpunkt	Luzie	Vincent
Vorstudie:			
Exposé	11.10.2021	70%	30%
Marktrecherche	08.10.2021	50%	50%
1. Audit			
Projektplan	15.10.2021	40%	60%
Architekturdiagramm	13.10.2021	30%	70%
Domänenmodell	13.10.2021	70%	30%
Alleinstellungsmerkmal	31.10.2021	70%	30%
Proof of concepts	20.10.2021	60%	40%
Erste Risiken	27.10.2021	50%	50%
Zielhierarchie	31.10.2021	30%	70%
Stakeholderanalyse	27.10.2021	50%	50%
Präsentation	03.11.2021	50%	50%

2. Audit			
Proof of concepts iteration	22.11.2021	50%	50%
Architekturdiagramm iteration	20.11.2021	30%	70%
Zielhierarchie iteration	20.11.2021	60%	40%
Risiken iteration	21.11.2021	60%	40%
Projektplan iteration	20.11.2021	30%	70%
Erste Struktur & Code	22.11.2021	40%	60%
Präsentation	1.12.2021	50%	50%
3. Audit			
Erster Prototype	19.12.2021	50%	50%
Durchgeführte PoCs	15.12.2021	50%	50%
Architekturdiagramm iteration	17.12.2021	30%	70%
Anwendungslogik diagramm	18.12.2021	40%	60%
Proof of concepts iteration	17.12.2021	70%	30%
Projektplan iteration	19.12.2021	30%	70%

Für den 3. Audit wurden die Iterationen des vorherigen Audits noch einmal aufgeführt, um anhand dieser und vorallem der schriftlich festgehaltenen PoC's den neuen Fortschritt erläutern zu können und die getätigten Schritte nachvollziehbar zu machen. Ziel war es vor allem, einen ersten Rapid Prototype zu programmieren, in dem die durchgeführten PoC's sichtbar miteinander interagieren. Außerdem wurden ein paar Artefakte iteriert, welche beim letzten Audit kritisiert wurden. Für den letzten Audit wird sich ebenfalls vorgenommen, gegebenenfalls Artefakte zu iterieren, eine kritische Erörterung des Projektes hinsichtlich der Zielhierarchie zu erstellen und den Main Prototype fertigzustellen.